

Data Structures & Algorithms

Stacks

Implementing a Stack

- Two different ways to implement a stack
 - Array
 - Linked list
- Which method to use depends on the application
 - what advantages and disadvantages does each implementation have?

Implementing a Stack Using Linked List

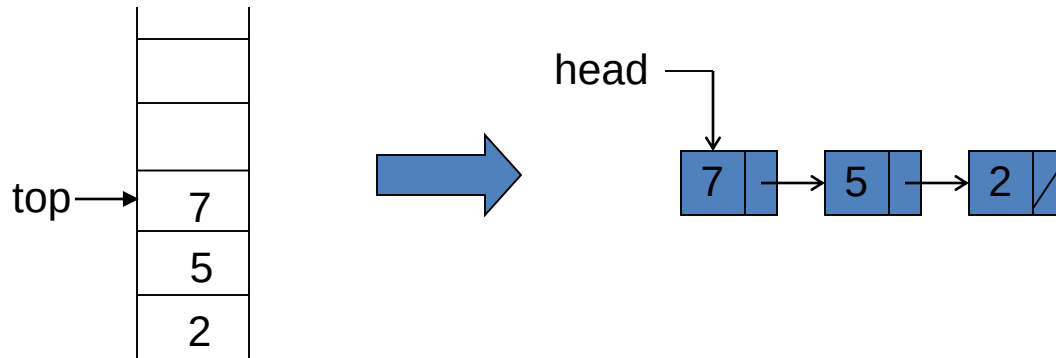
- Store the items in the stack in a linked list
- The top of the stack is the head node, the bottom of the stack is the end of the list
- *push* by adding to the front of the list
- *pop* by removing from the front of the list

Implementing a Stack: Linked List

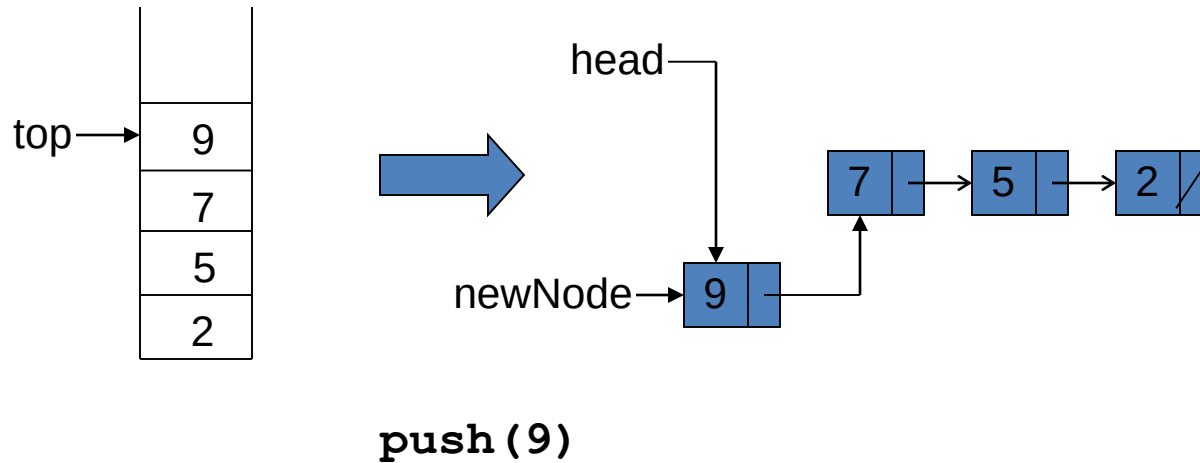
- Advantages:
 - always constant time to push or pop an element
 - can grow to an infinite size
- Basic implementation
 - list is initially empty
 - *push()* method adds a new item to the head of the list
 - *pop()* method removes the head of the list

Stack Using Linked List

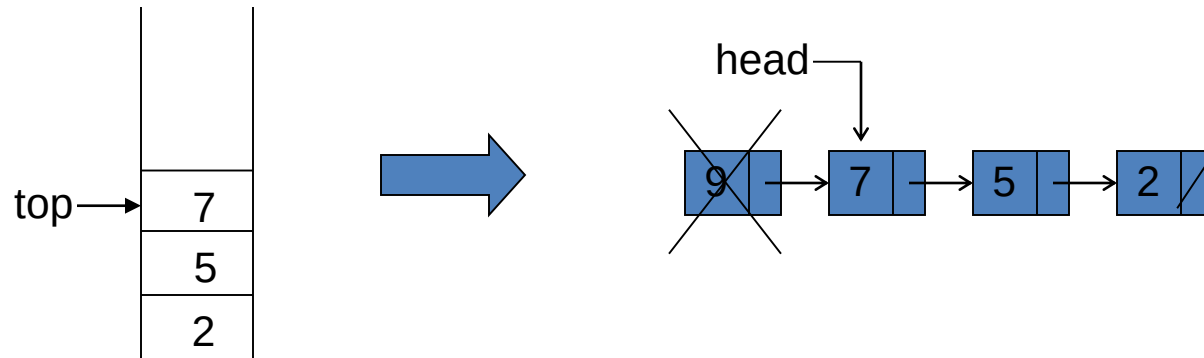
- No need for the current pointer; head is enough.



Stack Using Linked List: Push



Stack Using Linked List: Pop



IMPLEMENTATION

Stack Implementation Using Linked List

- We use two classes:
 - **Node**
 - **Stack**
- Declare `Node` class for the nodes
 - data: `int`-type data in this example
 - next: a pointer to the next node in the list

```
class Node {  
public:  
    int    data;           // data  
    Node*  next;           // pointer to next  
  
};
```

Stack Implementation Using Linked List

- Declare `Stack`, which contains
 - `head`: a pointer to the first node in the list i.e. top of the stack
Since the list is empty initially, `head` is set to `NULL`
 - Operations on Stack

```
class Stack{
public:
    Stack(void) // constructor
    {
        head = NULL;
    }
    ~Stack(void); // destructor
private:
    Node* head;

};
```

Stack Operations

- Operations of Stack
 - `IsEmpty`: determine whether or not the stack is empty i.e. `head == NULL`
 - `Push`: insert a new item at top of stack i.e. head position
 - `Pop`: delete an item from top of stack i.e. head position
 - `top`: return an item value from top of stack i.e. head position without deleting

Stack Implementation Using Linked List

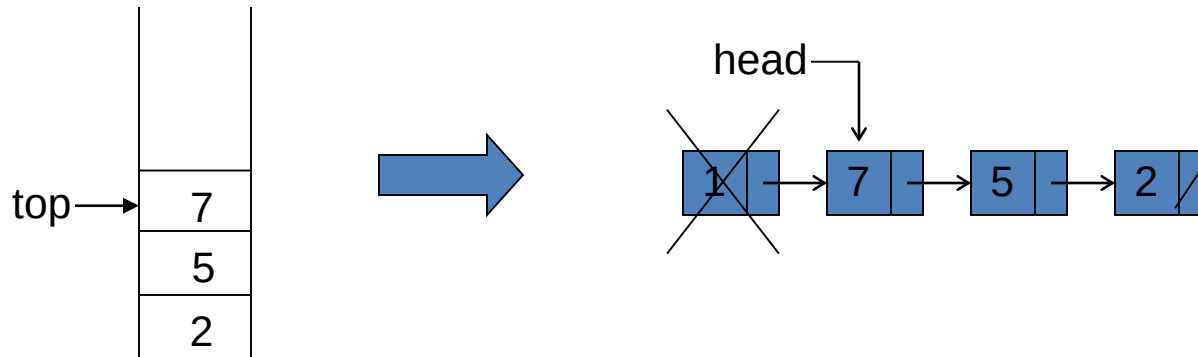
- Declare `Stack`, which contains
 - `head`: a pointer to the first node in the list i.e. top of the stack
Since the list is empty initially, `head` is set to `NULL`
 - Operations on Stack

```
class Stack{
public:
    Stack(void) // constructor
    {
        head = NULL;
    }
    ~Stack(void); // destructor
    void push(int x);
    int pop();
    int top();
    bool isEmpty();
private:
    Node* head;

};
```

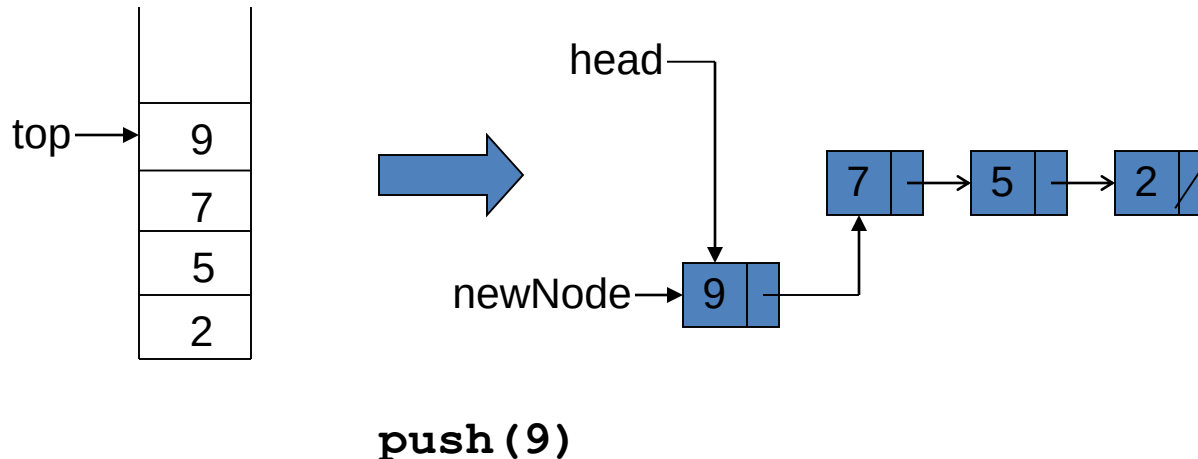
Pop Operation

```
int Stack::pop()
{
    int x = head->data;
    Node* p = head;
    head = head->next;
    delete p;
    return x;
}
```



Push Operation

```
void Stack::push(int x)
{
    Node* newNode = new Node();
    newNode->data = x;
    newNode->next = head;
    head = newNode;
}
```



Top Operation

```
int Stack::top()
{
    return head->data;
}
bool Stack::isEmpty()
{
    return ( head == NULL );
}
```

Uses of Stack

- Stacks are a very common data structure
 - compilers
 - parsing data between delimiters (brackets)
 - operating systems
 - program stack
 - virtual machines
 - manipulating numbers
 - pop 2 numbers off stack, do work (such as add)
 - push result back on stack and repeat
 - finding a path

Applications of Stacks

- Balancing Symbols
- Check the validity of a statement
- Eg: $(2 + 3 (4 - 5))$ is valid
- $(2 + 3 (4 - 5)$ is not valid

Applications of Stacks ctd..

- Evaluating Postfix expressions
- Consider the expression
$$(2 + 3 * (5 - 2) + 3 * 2)$$
- How do we write a program to find the answer to the above expression?
- Write the expression in postfix notation
$$2 \ 3 \ 5 \ 2 \ - \ * \ + \ 3 \ 2 \ * \ +$$
- Use a stack to evaluate it.

Example: postfix expressions

- Postfix notation is another way of writing arithmetic expressions.
- In postfix notation, the operator is written after the two operands.

infix: 2+5 postfix: 2 5 +

- Expressions are evaluated from left to right.
- Precedence rules and parentheses are never needed!!

Prefix, Infix, Postfix

- Two other ways of writing the expression are

+ A B	<i>prefix</i>
A B +	<i>postfix</i>

- The prefixes “pre” and “post” refer to the position of the operator with respect to the two operands.

Prefix, Infix, Postfix

- Consider the infix expression
$$A + B * C$$
- We “know” that multiplication is done before addition.
- The expression is interpreted as
$$A + (B * C)$$
- Multiplication has *precedence* over addition.

Prefix, Infix, Postfix

- Conversion to postfix

$A + (B * C)$ infix form

Prefix, Infix, Postfix

- Conversion to postfix

$A + (B * C)$

infix form

$A + (B C *)$

convert multiplication

Prefix, Infix, Postfix

- Conversion to postfix

$A + (B * C)$

infix form

$A + (B C *)$

convert multiplication

$A (B C *) +$

convert addition

Prefix, Infix, Postfix

- Conversion to postfix

$A + (B * C)$

infix form

$A + (B C *)$

convert multiplication

$A (B C *) +$

convert addition

$A B C * +$

postfix form

Prefix, Infix, Postfix

- Conversion to postfix

$(A + B) * C$ infix form

Prefix, Infix, Postfix

- Conversion to postfix

$(A + B) * C$

infix form

$(A B +) * C$

convert addition

Prefix, Infix, Postfix

- Conversion to postfix

$(A + B) * C$

infix form

$(A B +) * C$

convert addition

$(A B +) C *$

convert multiplication

Prefix, Infix, Postfix

- Conversion to postfix

$(A + B) * C$

infix form

$(A B +) * C$

convert addition

$(A B +) C *$

convert multiplication

$A B + C *$

postfix form

Precedence of Operators

- The five binary operators are: addition, subtraction, multiplication, division and exponentiation.
- The order of precedence is (highest to lowest)
- Exponentiation $\uparrow$
- Multiplication/division $*, /$
- Addition/subtraction $+, -$

Precedence of Operators

- For operators of same precedence, the left-to-right rule applies:

$A+B+C$ means $(A+B)+C$.

- For exponentiation, the right-to-left rule applies

$A \uparrow B \uparrow C$ means $A \uparrow (B \uparrow C)$

Infix to Postfix

Infix

$A + B$

$12 + 60 - 23$

$(A + B) * (C - D)$

$A \uparrow B * C - D + E / F$

Postfix

$A B +$

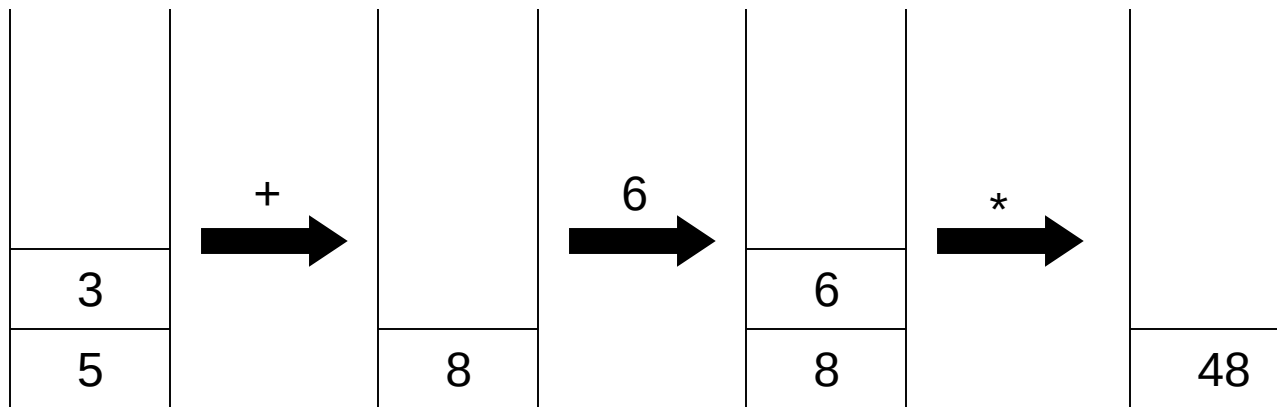
$12 60 + 23 -$

$A B + C D - *$

$A B \uparrow C * D - E F / +$

Reverse Polish Notation

- Way of inputting numbers to a calculator
 - $(5 + 3) * 6$ becomes $5\ 3\ +\ 6\ *$
 - $5 + 3 * 6$ becomes $5\ 3\ 6\ * +$
- We can use a stack to implement this
 - consider $5\ 3\ +\ 6\ *$



- try doing $5\ 3\ 6\ * +$

Evaluating Postfix

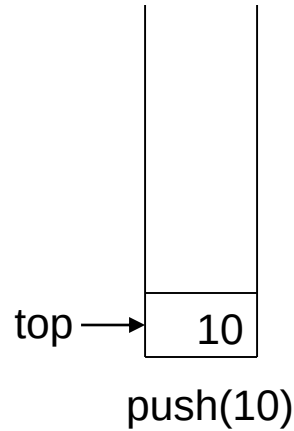
```
Stack s;  
while( not end of input ) {  
    e = get next element of input  
    if( e is an operand )  
        s.push( e );  
    else {  
        op2 = s.pop();  
        op1 = s.pop();  
        value = result of applying operator 'e' to op1 and op2;  
        s.push( value );  
    }  
}  
finalresult = s.pop();
```

Evaluating Postfix

- 10 2 8 * + 3 -

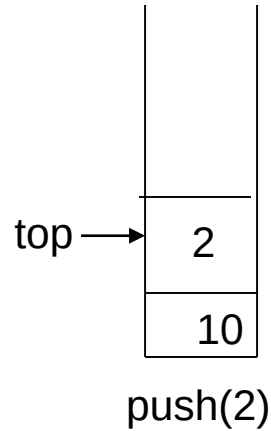
Evaluating Postfix: 10 2 8 * + 3 -

- First, push(10) into the stack



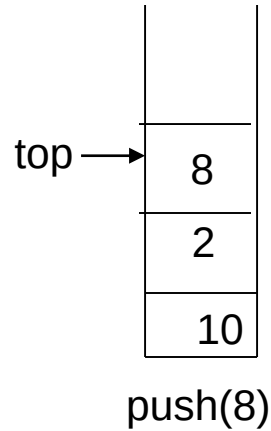
Evaluating Postfix: 10 2 8 * + 3 -

- Then, push(2) into the stack



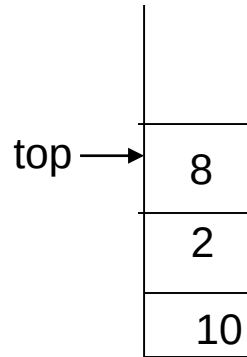
Evaluating Postfix: 10 2 8 * + 3 -

- Then, push(8) into the stack



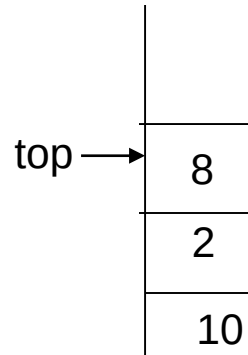
Evaluating Postfix: 10 2 8 * + 3 -

- Now we see an operator *, that means we can get a new number by calculation



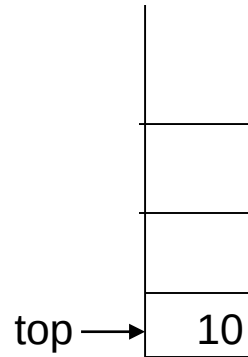
Evaluating Postfix: 10 2 8 * + 3 -

- Now we see an operator *, that means we can get a new number by calculation
- Pop the first two numbers



Evaluating Postfix: 10 2 8 * + 3 -

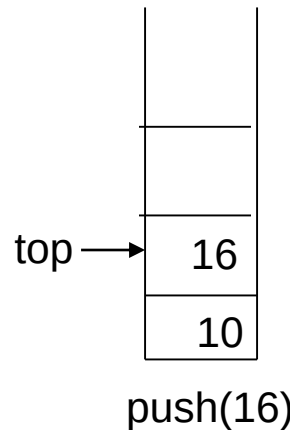
- Now we see an operator *, that means we can get a new number by calculation
- Pop the first two numbers



$$8 * 2 = 16$$

Evaluating Postfix: 10 2 8 * + 3 -

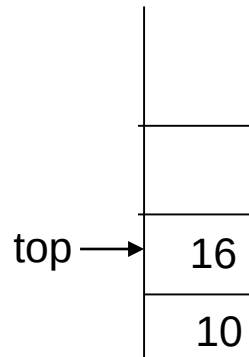
- Now we see an operator *, that means we can get a new number by calculation
- Pop the first two numbers
- Push the new number



$$8 * 2 = 16$$

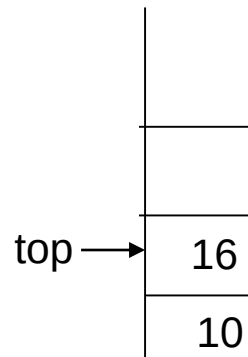
Evaluating Postfix: 10 2 8 * + 3 -

- Then we see the next operator + and perform the calculation



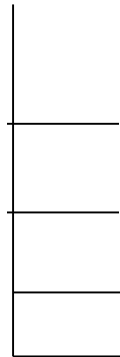
Evaluating Postfix: 10 2 8 * + 3 -

- Then we see the next operator + and perform the calculation
- Pop the two numbers



Evaluating Postfix: 10 2 8 * + 3 -

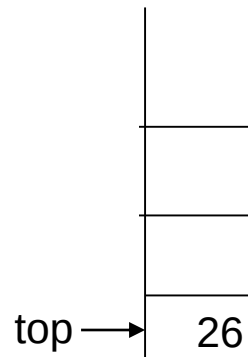
- Then we see the next operator + and perform the calculation
- Pop the two numbers



- $16 + 10 = 26$

Evaluating Postfix: 10 2 8 * + 3 -

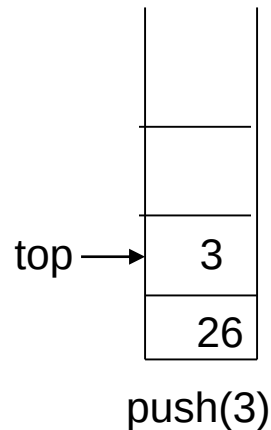
- Then we see the next operator + and perform the calculation
- Pop the two numbers
- Push the new number



- $16 + 10 = 26$

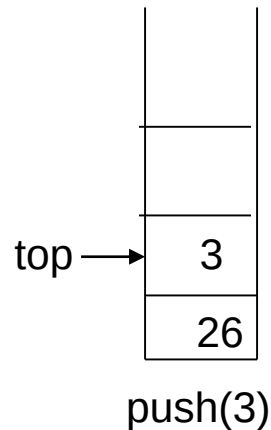
Evaluating Postfix: 10 2 8 * + 3 –

- Now we see the number 3, push(3) into the stack



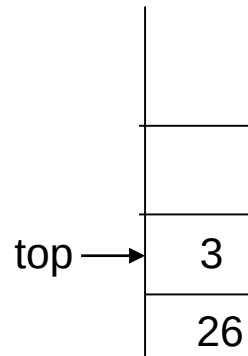
Evaluating Postfix: 10 2 8 * + 3 -

- Now we see the operator -



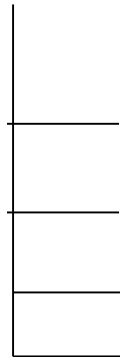
Evaluating Postfix: 10 2 8 * + 3 –

- Now we see the operator –
- Pop the two numbers



Evaluating Postfix: 10 2 8 * + 3 -

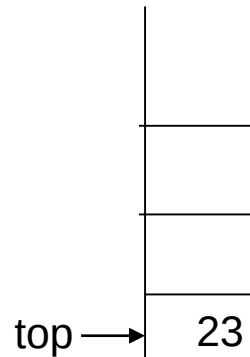
- Now we see the operator -
- Pop the two numbers



- $26 - 3 = 23$

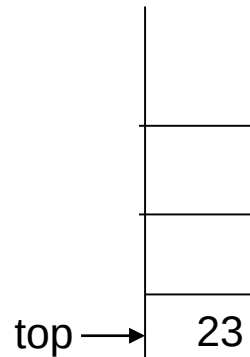
Evaluating Postfix: 10 2 8 * + 3 –

- Now we see the operator –
- Pop the two numbers
- Push the new number



Evaluating Postfix: 10 2 8 * + 3 -

- End of input, pop the stack i.e. answer



Converting Infix to Postfix

- Consider the infix expressions ' $A+B*C$ ' and ' $(A+B)*C$ '.
- The postfix versions are ' $ABC*+$ ' and ' $AB+C*$ '.
- The order of operands in postfix is the same as the infix.
- In scanning from left to right, the operand ' A ' can be inserted into postfix expression.

Converting Infix to Postfix

- The '+' cannot be inserted until its second operand has been scanned and inserted.
- The '+' has to be stored away until its proper position is found.
- When 'B' is seen, it is immediately inserted into the postfix expression.
- Can the '+' be inserted now? In the case of 'A+B*C' cannot because * has precedence.

Converting Infix to Postfix

- In case of $(A+B)*C$, the closing parenthesis indicates that $+$ must be performed first.
- Assume the existence of a function $\text{prcd}(\text{op1}, \text{op2})$ where op1 and op2 are two operators.
- $\text{Prcd}(\text{op1}, \text{op2})$ returns TRUE if op1 has precedence over op2 , FALSE otherwise.

Converting Infix to Postfix (1)

- $\text{prcd}('*', '+')$ is TRUE
- $\text{prcd}('+', '+')$ is TRUE
- $\text{prcd}('+', '*')$ is FALSE
- $\text{prcd}('+', '-')$ is TRUE
- Here is the algorithm that converts infix expression to its postfix form.
- The infix expression is without parenthesis.
- Next approach (2) shows algorithm for conversion of infix to postfix with parenthesis.

Converting Infix to Postfix (1)

```
1.  Stack s;
2.  While( not end of input ) {
3.      c = next input character;
4.      if( c is an operand )
5.          add c to postfix string;
6.      else {
7.          while( !s.empty() && prcd(s.top(),c) ){
8.              op = s.pop();
9.              add op to the postfix string;
10.         }
11.         s.push( c );
12.     }
13. }
14. while( !s.empty() ) {
15.     op = s.pop();
16.     add op to postfix string;
17. }
```

Converting Infix to Postfix (2)

```
// function to return precedence value
// if operator is present in stack
int inPrec(char input)
{
    switch (input) {
        case '+':
        case '-':
            return 2;
        case '*':
        case '%':
        case '/':
            return 4;
        case '(':
            return 0;
        case '^':
            return 5;
    }
}
```

```
// function to return precedence value
// if operator is present outside stack.
int outPrec(char input)
{
    switch (input) {
        case '+':
        case '-':
            return 1;
        case '*':
        case '%':
        case '/':
            return 3;
        case '(':
            return 100;
        case '^':
            return 6;
    }
}
```

Converting Infix to Postfix (2)

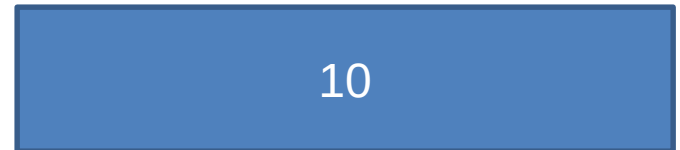
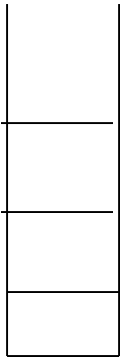
```
1.  Stack s;
2.  While( not end of input ) {
3.      c = next input character;
4.      if( c is an operand )
5.          add c to postfix string;
6.      else {
7.          while( !s.empty() && inPrec(s.top())>outPrec(c) ){
8.              op = s.pop();
9.              add op to the postfix string;
10.         }
11.         s.push( c );
12.     }
13. }
14. while( !s.empty() ) {
15.     op = s.pop();
16.     add op to postfix string;
17. }
```

Transforming Infix to postfix

- Ex: $10 + 2 * 8 - 3$

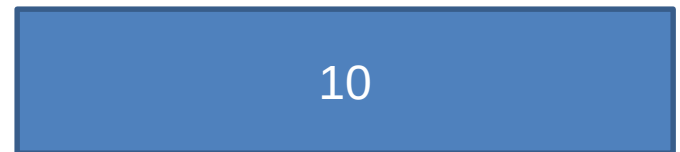
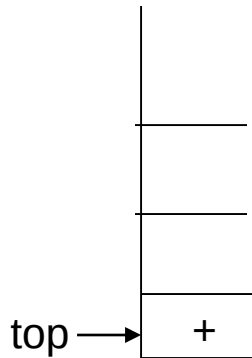
Transforming Infix to postfix

- Ex: **10** + 2 * 8 - 3
- We see the first number 10, output it



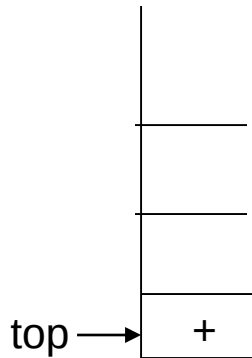
Transforming Infix to postfix

- Ex: $10 + 2 * 8 - 3$
- We see the operator $+$, push it into the stack



Transforming Infix to postfix

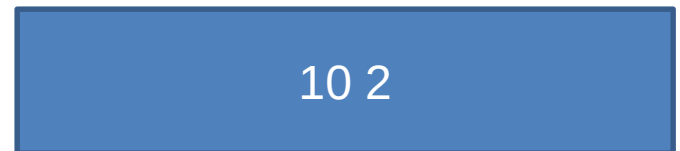
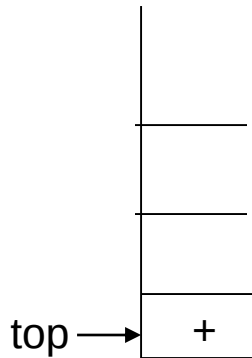
- Ex: $10 + 2 * 8 - 3$
- We see the number 2, output it



10 2

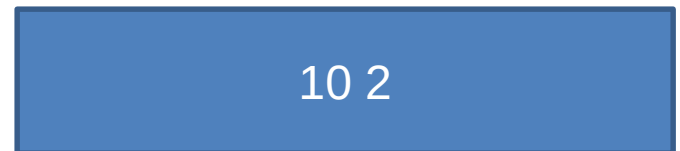
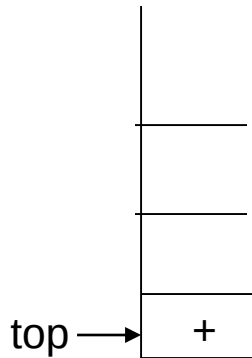
Transforming Infix to postfix

- Ex: $10 + 2 * 8 - 3$
- We see the operator $*$



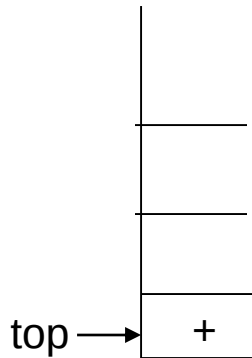
Transforming Infix to postfix

- Ex: $10 + 2 * 8 - 3$
- We see the operator $*$
- Check the precd/priority of the top operator



Transforming Infix to postfix

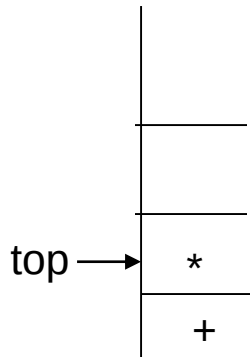
- Ex: $10 + 2 * 8 - 3$
- We see the operator $*$
- Check the precd/priority of the top operator
- $*$ precd is higher than $+$, push($*$)



10 2

Transforming Infix to postfix

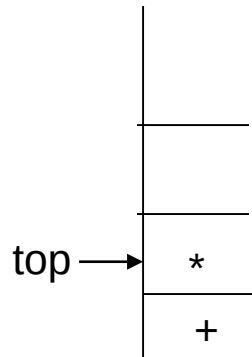
- Ex: $10 + 2 * 8 - 3$
- We see the operator $*$
- Check the precd/priority of the top operator
- $*$ precd is higher than $+$, push($*$)



10 2

Transforming Infix to postfix

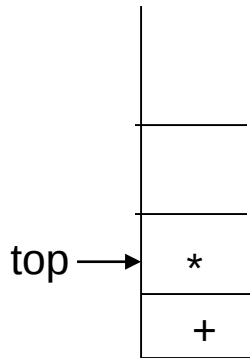
- Ex: $10 + 2 * 8 - 3$
- We see the number 8, output it



10 2 8

Transforming Infix to postfix

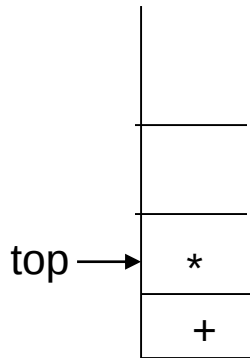
- Ex: $10 + 2 * 8 - 3$
- We see the operator $-$
- Check the precd/priority of the top operator



10 2 8

Transforming Infix to postfix

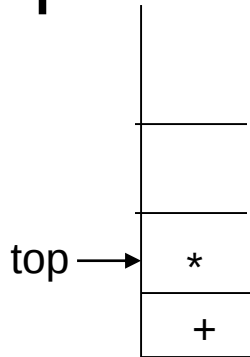
- Ex: $10 + 2 * 8 - 3$
- We see the operator $-$
- Check the precd/priority of the top operator
- - precd is less than $*$



10 2 8

Transforming Infix to postfix

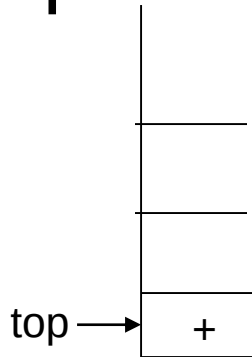
- Ex: $10 + 2 * 8 - 3$
- We see the operator $-$
- Check the precd/priority of the top operator
- - precd is less than $*$
- pop



10 2 8

Transforming Infix to postfix

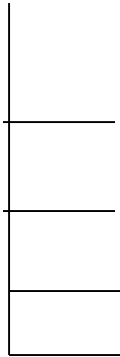
- Ex: $10 + 2 * 8 - 3$
- We see the operator $-$
- Check the precd/priority of the top operator
- - precd is less than $*$
- Pop



10 2 8 *

Transforming Infix to postfix

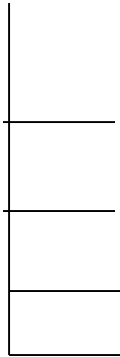
- Ex: $10 + 2 * 8 - 3$
- We see the operator $-$
- Check the precd/priority of the top operator
- - precd is less than $*$
- pop



10 2 8 * +

Transforming Infix to postfix

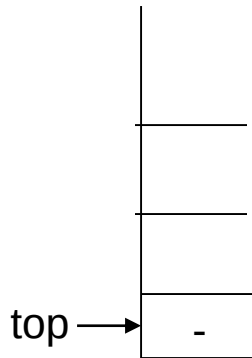
- Ex: $10 + 2 * 8 - 3$
- We see the operator $-$
- Check the precd/priority of the top operator
- $-$ precd is less than $*$
- pop
- Push($-$)



10 2 8 * +

Transforming Infix to postfix

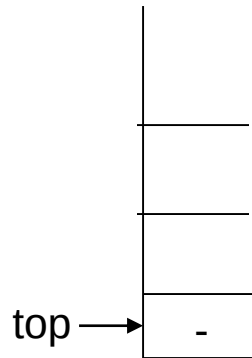
- Ex: $10 + 2 * 8 - 3$
- We see the operator $-$
- Check the precd/priority of the top operator
- $-$ precd is less than $*$
- pop
- Pus($-$)



10 2 8 * +

Transforming Infix to postfix

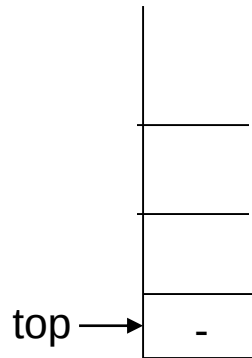
- Ex: $10 + 2 * 8 - 3$
- We see the number 3, output it



10 2 8 * +3

Transforming Infix to postfix

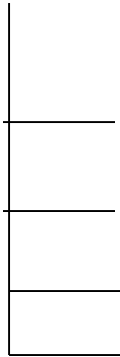
- Ex: $10 + 2 * 8 - 3$
- Expression is ended, pop all operators present in stack



10 2 8 * +3

Transforming Infix to postfix

- Ex: $10 + 2 * 8 - 3$
- Expression is ended, pop all operators present in stack



10 2 8 * +3 -

THANK YOU